SoK: Authorization for Autonomous Agents

Anonymous Author(s)

ABSTRACT

An autonomous agent calls tools and Model Context Protocol (MCP) servers while carrying a bearer token (an OAuth 2.0 access token or an MCP authorization token) on each request. Because the agent is driven by untrusted input, its authorization layer is part of the attack surface: a token not bound to the server it is sent to can be replayed or passed through, an over-broad scope hands an injected agent far more than any task needs, and an on-behalf-of token with no verifiable delegation chain leaves "who authorized this?" unanswerable. A fast-growing research literature studies agent security, but its coverage is densest on model-level attacks (prompt injection, tool poisoning) and it proposes individual mechanisms; it does not systematize the authorization layer. We survey 44 research works and show that two authorization properties, audience binding and confused-deputy / dynamic client registration, are treated as mere specification obligations rather than as properties anyone tests or enforces. We then systematize agent authorization along three axes no prior work combines: (i) by *pre-deployment testability*, meaning which threats are decidable from a token, a key set, and a declared tool contract before the agent runs; (ii) by *normative provenance*, a threat-to-clause mapping to the OAuth RFCs, the MCP authorization specification, OWASP, and CWE; and (iii) by a *three-way enforcement spectrum*, token-static vs. server-dynamic vs. needing a human or the model. Across fifteen threats in six categories, we find twelve are testable before production, six from a token alone and six against a staging server with no human or model in the loop, and only three need a human or the model. We evaluate the token-static subset against nine independent verifiers, eight libraries across three ecosystems and the Envoy proxy, and find six accept a token they should reject, a no-expiry token (four, including the Envoy proxy that fronts service meshes) or an unbound-audience token (one), so the weaknesses are defaults, not corner cases. We argue this cut should guide both tooling and standardization.

1 INTRODUCTION

When an agent calls a tool or an MCP server on a user's behalf, a bearer token travels with the request. The failure modes are OAuth failure modes, but with sharper consequences, because the caller is autonomous and can be steered by untrusted input such as a retrieved document or a tool description [11]. A token not bound to the resource it is presented to can be replayed against, or forwarded through, a different server (a confused deputy); an on-behalf-of (OBO) token with no verifiable delegation chain makes accountability impossible; an omnibus scope such as `admin:*` hands an injected agent the keys to everything; and a non-expiring credential never stops being useful to whoever obtains it.

Conference'17, July 2017, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

What agent autonomy and MCP change. Each failure above is a known OAuth or JWT weakness, and the individual checks that catch them, require an `aud`, cap a scope, demand an `exp`, are classical hygiene. Two things are new. First, the client is now autonomous and driven by untrusted input, so a single over-broad or unbound token is not merely a misconfiguration a careful client avoids but a live capability an injected plan will exercise: the token is the attack surface. Second, MCP composes what single-service OAuth did not, and the composition is where authorization breaks: tokens pass through chains of servers, delegation nests across agent and sub-agent, clients register dynamically, and a server publishes a declared tool contract against which a token's scope can finally be judged. Our contribution is therefore not the individual checks but the systematization they enable under this threat model, sorting the threats by who can test each, and before what.

A large and fast-moving body of research now studies agent and MCP security. As our survey (§6) shows, it is dense on *model-level* attacks (prompt-injection-driven escalation and tool poisoning) and it proposes many *individual* defense mechanisms (privilege controllers, delegation protocols, identity architectures). What it does not do is systematize the *authorization* layer itself: no surveyed work sorts agent-authorization threats by whether a defender can test them before deployment, maps each threat to the specific normative clause it violates, or names the line between what can be enforced statically at the trust boundary and what can only be enforced at runtime. The two authorization properties that matter most for token safety, audience binding and the confused-deputy / dynamic-client-registration pair, are treated across 44 works as specification obligations to cite rather than as properties to verify (§6).

Contributions. (i) We give a source-grounded threat model of fifteen agent-authorization threats in six categories, each with its broken invariant, the conditions under which it is exploitable, a mitigation, and its normative source (§2, §3). (ii) We systematize this space along three axes no prior work combines: testability, normative provenance, and a three-way enforcement spectrum, token-static vs. server-dynamic vs. human-or-model; by it, twelve of fifteen threats are testable before production and only three need a human or the model (§4). (iii) We survey 44 research works on agent authorization and make the coverage gap precise: audience binding and confused-deputy are the under-formalized seam (§6). (iv) We argue which authorization properties a standard or a pre-deployment tool could mandate, release an open-source analyzer for the token-static subset (§4), and evaluate it against nine reference verifiers, six of which accept a token they should reject (§5).

2 BACKGROUND

Tokens and claims. An OAuth 2.0 access token authorizes a client to call a resource on a user's behalf; in agent deployments it is usually a JSON Web Token (JWT), a signed set of claims. The claims

that govern authorization are the issuer (`iss`), the intended audience (`aud`, the resource the token is *for*), the granted `scope`, the expiry (`exp`), and, for delegation, the authorized party (`azp`, naming the client) and the actor (`act`, naming who is acting). A verifier must check the signature against the issuer's keys, that `aud` names itself, that the algorithm is on an allow-list, and that required claims hold (RFC 8725). RFC 9068 profiles JWT access tokens and makes audience and issuer validation mandatory for that profile; RFC 8707 (resource indicators) lets a client request a token bound to a specific resource.

Delegation and the `act`/`azp` claims. OAuth token exchange (RFC 8693) issues a delegated token carrying an `act` claim that names the acting party and a `may_act` claim stating who is *permitted* to act for the subject. A single `act` claim records one hop; a chain (user → agent → sub-agent) requires *nested* `act` claims, and transitive delegation is only verifiable if each exchange enforces `may_act` and the verifier walks the full nested chain. `azp` names the client that obtained the token but does not by itself establish a delegation chain: it identifies the immediate party, not the sequence of authorizations behind it. This is why `azp`/`act` alone are necessary but not sufficient: without enforced `may_act` and nested-chain validation, "who authorized this?" remains unanswerable across more than one hop.

MCP, identity, and the confused deputy. The Model Context Protocol standardizes how an agent discovers and calls tools; its authorization specification inherits OAuth and *prohibits* passing a client's token through to an upstream API. Dynamic client registration (DCR, RFC 7591) lets clients register at runtime, which an MCP proxy may expose. Workload-identity substrates (WIMSE; SPIFFE/SPIRE) would give each agent instance a verifiable, short-lived identity distinct from the human's, but are rarely deployed. The confused-deputy problem (CWE-441), where a more privileged party misuses its authority on behalf of a less privileged one, is the classical frame for token passthrough and consent-flow abuse.

The agent trust model. Two properties of the agent setting sharpen every OAuth failure above. First, the caller is *autonomous*: it chooses which tool to call and with which arguments, so a credential it holds is exercised without a human reviewing each use. Second, its inputs are *untrusted*: an agent reads documents, web pages, and tool descriptions an attacker may control, so prompt injection can steer it into requests the user never intended [11]. Together these mean the agent must be treated as a party that can be induced to misuse whatever its token permits, the assumption our threat model adopts and that recent multi-agent security agendas make explicit [39]. The consequence for authorization is concrete: the token an agent carries, not a human's judgement at call time, is the last line of defense, so the properties that token enforces are what bound the blast radius of an injected agent. This is why we treat the token-level, statically checkable properties as first-class rather than as spec footnotes.

3 A THREAT MODEL FOR AGENT AUTHORIZATION

We organize the space into six categories. Table 1 lists fifteen threats with the invariant that must hold, a one-line mitigation, and the weakest harness that can decide the threat before production (its Test class, §4). We then give, per category, the exploit condition and the defense.

Attacker and defender capabilities. Two capabilities must be kept apart, because conflating them is easy and misleads. The *attacker* we model steers the agent through untrusted input, a poisoned document or a malicious tool description, and may capture a token in transit; it does not hold the issuer's signing key, so it cannot mint a token with an arbitrary `aud`, strip `exp`, or forge an `act` chain. What it exploits is a token the deployment already accepts: an over-broad scope it can cash in (SCOPE-01), an unbound or shared-issuer audience it can replay or pass through (MCP-01/02), a missing delegation record that defeats attribution (DEL-01). The *defender*, by contrast, red-teams with issuer control in a test harness: it mints the adversarial token an attacker would benefit from and checks that its own verifier rejects it. The testability classes of §4 describe this *defender* capability, not the attacker's: a threat is token-static when the defender can settle it from a minted token offline, whether or not a real attacker could have produced that token unaided.

Audience binding (MCP-01/02). A bearer token names, in its `aud` claim, the resource it is for. The exploit has two shapes. In *passthrough* (MCP-01), an MCP server that receives a client's token forwards it unchanged to an upstream API; because the upstream often shares an issuer it accepts the token, and the MCP server has acted as a confused deputy wielding the full authority of the user's token. In *replay* (MCP-02), a token minted with no resource indicator, or one whose `aud` a verifier never checks, is accepted at any server the issuer serves, so an attacker who captures one token reaches many. Both are properties of claims the token carries and of the server's declared identity, so a defender mints the adversarial token and confirms rejection before deployment. The defense is to request resource-bound tokens (RFC 8707), to verify `aud` names this server, and to honor the MCP prohibition on passthrough. Among the works we survey only Agentic JWT [10] and one enterprise framework [30] treat audience binding as a property to enforce rather than a specification clause to cite (§6).

Delegation (DEL-01..04). When an agent acts for a user, an on-behalf-of (OBO) token should carry an `act` claim naming the acting party (RFC 8693). A single `act` records one hop; a chain user → agent → sub-agent needs *nested* `act` claims. A caveat governs what any inspection of them can conclude. RFC 8693 defines the `act` claim, including its nested prior actors, as *informational*; it is a record of identity, not an authorization input, and must not on its own drive an access-control decision. A signed `act` history therefore shows *who* acted but does not prove that each hop was *authorized*; that holds only if every token exchange along the chain enforced the `may_act` claim at issuance. The exploit: with a missing or malformed `act` (DEL-01) an action cannot be attributed, and with a chain whose `may_act` was never enforced (DEL-02) a sub-agent silently exceeds what was delegated, a transitive-trust escalation. What a presented token settles is thus a hygiene property: a verifier confirms from the token alone that a required `act` claim is present and the nested chain is well-formed, and rejects a token that asserts delegation but carries none. Confirming that the chain was *authorized* is an extension

Table 1: Agent-authorization threats by category, with the invariant that must hold, a mitigation, the normative source, and the *weakest harness* that can decide the threat before production (Test): T, from a token, key set, and declared contract alone; S, against a staging server or issuer with no human or model in the loop; H, only with a live human approver or the model itself. Sev.: severity (critical/high/med, per §6).

ID	Sev.	Broken invariant (exploit)	Mitigation	Source	Test
Audience binding					
MCP-01	crit	token accepted with wrong/any audience (passthrough, replay)	verify `aud` names this resource; reject forwarded tokens	RFC 8707/9728; MCP [1, 2, 17]	T
MCP-02	high	token carries no resource indicator (unbound audience)	request resource-bound tokens; require `aud`	RFC 8707 [2]	T
Delegation					
DEL-01	high	missing or forged `act` claim (no accountable actor)	require a signed `act` claim on OBO tokens	RFC 8693 [18]	T
DEL-02	high	unverifiable nested delegation chain (transitive-trust abuse)	validate the full nested `act` chain	RFC 8693 [18]	T
DEL-03	high	`may_act` not enforced on exchange	enforce `may_act` at the token endpoint	RFC 8693 [18]	S
DEL-04	med	impersonation where delegation was required	issue delegated (not impersonated) tokens	RFC 8693 [18]	S
Scope					
SCOPE-01	high	over-scoped agent credential (`admin:*`)	least-privilege scopes per task	RFC 9700; OWASP [23, 33]	T
SCOPE-02	high	non-expiring / long-lived token	short `exp`; rotation	RFC 8725 [41]	T
SCOPE-03	high	agent reuses the human's root credential	distinct agent credential	OWASP [33]	S
Agent identity					
AID-01	high	session used as authentication	bind to a verifiable workload identity	WIMSE/SPIFFE [15, 45]	S
AID-02	med	no distinct workload/agent identity	issue per-agent SVID/credential	WIMSE/SPIFFE [15, 45]	S
Confused deputy					
CD-01	crit	proxy acts on a client's behalf without per-client consent	per-client consent; exact `redirect_uri` match	CWE-441; MCP [1, 28]	H
CD-02	high	dynamic client registration abuse	vet/limit DCR; bind registrations	RFC 7591 [37]	S
Agency					
AGT-01	crit	prompt-injection-driven privilege escalation	per-action authorization; human-in-loop	OWASP [32, 33]	H
AGT-02	high	tool poisoning / rug-pull via tool metadata	vet tool descriptions; pin versions	OWASP [33]	H

resting on an explicit trust assumption, that the issuer enforced `may_act`, and cannot be read off the token. The remaining two are plainly runtime: whether the token endpoint enforced `may_act` on each exchange (DEL-03), and whether a delegated rather than an impersonated token was issued (DEL-04), are properties of the issuing flow, not of the presented token. Delegation protocols that bind each action to user intent and bound a sub-agent's authority target this category [10, 29, 34, 47]; an empirical gap analysis finds identity, authorization, and runtime governance thinly deployed across multi-agent systems in practice [4].

Scope (SCOPE-01..03). The `scope` and `exp` a token carries bound what an injected agent can do with it. The exploit: an omnibus scope such as `admin:*` (SCOPE-01) hands an injected agent far more than any task needs, and a non-expiring token (SCOPE-02) stays useful to whoever later obtains it. Both are read directly from the token, so a check against a declared allow-list and a maximum lifetime is a pre-deployment test. The third, an agent reusing the human's root credential (SCOPE-03), is a property of how the deployment provisions credentials, not of any single token, and so is runtime. Least-privilege frameworks enforce the scope constraint at runtime with monotonic confinement [43], and measurements show over-privilege is common in practice [22]; our point is that the carried scope and lifetime are checkable before the agent ever runs.

Agent identity (AID-01/02). An agent needs a verifiable identity distinct from the human's so its actions can be attributed and its access revoked. The exploit: treating a session token as authentication (AID-01), or giving the agent no workload identity at all (AID-02), leaves no principal to revoke or audit. The defense is a deployed workload-identity substrate (WIMSE, SPIFFE/SPIRE) that issues each agent instance a short-lived, verifiable credential [31, 40, 46]. Both are runtime: with no identity layer deployed there is nothing in a presented token to check against, which is why they sit on the runtime side of the line.

Confused deputy (CD-01/02). An OAuth proxy sits between many clients and a resource. The exploit: if the proxy reuses one consent across clients, or matches `redirect_uri` loosely, one client inherits another's authorization (CD-01); and if it exposes dynamic client registration without vetting (RFC 7591), an attacker registers a client the proxy then trusts (CD-02). These are properties of a live consent flow and of how the proxy matches redirects at request time, so they cannot be decided from a token or a static manifest. Protocol-level analyses of MCP surface exactly these trust-propagation gaps [21, 25], yet across the literature the confused deputy appears almost only as a named taxonomy entry, not an enforced property (§6).

Agency (AGT-01/02). The agency threats are where authorization meets the model. The exploit: untrusted input drives the agent to escalate privilege (AGT-01) or to call a tool whose description has been poisoned (AGT-02), so the agent itself issues the malicious request with its own legitimate token. No token check helps, because the token is valid and the request is the agent's own; the decision

depends on the executing context. The defenses are runtime: per-action authorization or a human in the loop so an injection cannot escalate on its own [43], and vetting of tool descriptions before they reach the model [36, 54]. This category is the most densely studied in the literature (§6), which is part of our point: the field has concentrated on the model-level attacks and under-formalized the token-level guarantees.

A worked scenario. Consider a travel-planning agent that books flights through an MCP server, which in turn calls an airline API. The user authenticates once; the agent then receives an access token and acts autonomously. Three failures from Table 1 compose. If the booking server forwards the user's token to the airline API rather than exchanging it (MCP-01), and the airline shares the issuer, a malicious or compromised airline endpoint replays the user's token against any resource that issuer serves. If the token carries scope `travel:*` rather than `flights:book` (SCOPE-01), a prompt injected through a hotel-review page the agent reads can drive it to cancel bookings or read the user's itinerary history, because the token already permits it. And if the delegated token carries no `act` claim (DEL-01), neither the airline nor an auditor can later establish that the agent, rather than the user directly, issued the request. Each of these three is visible in the token and the server's declared contract *before the agent runs*: a defender mints a token with the wrong audience, an over-broad scope, and a missing `act`, and confirms the verifier rejects each. The per-client human consent the airline relied on (CD-01) and the prompt injection that triggered the misuse (AGT-01) are not visible this way; they need a live human approver and a running agent. The same scenario therefore straddles the line, and the line tells the team which part to gate in CI, which part to test against staging, and which part needs a runtime control.

4 SYSTEMATIZING THE SPACE

Our systematization rests on three axes that, together, no prior work applies to agent authorization (§6).

(1) Pre-deployment testability. Testability before an agent runs is not one thing but a short spectrum, and collapsing its halves is a common imprecision we make explicit. We separate three modes. *Token inspection* decides a property from a presented token and the issuer's key set alone: whether an `aud` is present and names this server, whether `scope` lies within a declared allow-list, whether `exp` is set and within a cap, and whether a required `act` claim is present and well-formed. *Conformance testing* mints the adversarial token an attacker would and confirms that the *deployed verifier* rejects it; it runs before the agent is exercised but, unlike token inspection, needs the server's enforcement path, because whether a server honors an `aud` or refuses passthrough is a property of its implementation, not of the token. *Runtime monitoring* (H) needs a live human approver or the model itself. These three modes, not a binary, are what Table 1's Test column records, and by them the fifteen threats sort **6/6/3**. Six are token-static (T): audience binding (MCP-01/02), the delegation *hygiene* checks (DEL-01/02, in the bounded sense of §3), and scope and expiry (SCOPE-01/02). Six more are server-dynamic (S): whether the issuer enforces `may_act` or issues delegation rather than impersonation (DEL-03/04), whether the agent is provisioned a distinct credential (SCOPE-03), whether a workload-identity substrate attests each agent (AID-01/02), and whether dynamic client registration is left open (CD-02). Each of the six is testable against a staging server or issuer with no human and no model in the loop; a binary static/runtime split understates this by filing them all under "runtime." Only three are irreducibly H: per-client human consent (CD-01) and the two agency threats, prompt-injection escalation and tool poisoning (AGT-01/02), which turn on the model's behavior under untrusted input. The result is sharper than a two-way line suggests: twelve of fifteen threats are testable before production, six from a token and six against a staging server, and only three need a human or the model.

(2) Normative provenance. Each threat maps to the clause it violates: audience binding to RFC 8707 and the MCP passthrough prohibition; delegation to RFC 8693; scope and expiry to RFC 9700 and RFC 8725; registration to RFC 7591; confused deputy to CWE-441; agency to the OWASP LLM and Agentic Top-10s [32, 33]. This lets a practitioner trace a defense to the authority that mandates it, a mapping the surveyed literature does not provide.

(3) The enforcement spectrum. The defenses in the literature scatter across a spectrum they never name: *static* enforcement at the trust boundary (control/data-flow extraction [5], capability attestation, contract checks on tokens and key sets) versus *runtime* enforcement during execution (monotonic privilege confinement [43], pre-action authorization [48], mandatory access control [16], out-of-band policy [27]). Drawing the line explicitly reframes several "runtime-only" defenses as compensating controls for threats a pre-deployment contract check could have excluded: if audience binding and scope are verified statically, a runtime controller has strictly less to contain. Figure 1 places the fifteen threats on the three-way spectrum this yields, T/S/H.

An instrument for the token-inspection subset. We release an open-source analyzer that performs the token-inspection checks of mode one: it decodes a presented token against a policy and reports cited findings on audience binding, scope, expiry, and the delegation hygiene property (a required `act` claim present and well-formed). It is explicit about the boundary the three modes draw. It *verifies* the properties a token and key set settle; it only *decodes and reports* the rest, because, exercising no server, it cannot attest that a deployment refuses passthrough or enforces `may_act`. That attestation is conformance testing, for which MCP security benchmarks and harnesses are the right instrument [12, 53] and which our token-level checks complement rather than replace. The analyzer is stateless and makes no network calls, so it runs in CI or beside the agent. It covers the six token-static threats; the six server-dynamic threats need a conformance harness against a staging server, and the three human-or-model threats a runtime control. Naming that boundary is itself the result: it states what a pre-deployment token check, and separately a conformance suite and a tightened standard, can and cannot guarantee.

5 EVALUATION: DO REAL VERIFIERS REJECT THE TOKENS?

The server-dynamic claim is testable, so we tested it. We stood up nine independent verifiers: eight widely-used libraries spanning

T token: a token, key set, contract	**S staging server**: no human, no model	**H human / model**
MCP-01, MCP-02	DEL-03, DEL-04	CD-01
DEL-01, DEL-02	SCOPE-03, CD-02	AGT-01
SCOPE-01, SCOPE-02	AID-01, AID-02	AGT-02
6: testable from a token	*6: testable against staging*	*3: need a human/model*

Figure 1: The testability spectrum. Twelve of fifteen threats are testable before production, six from a token alone (T) and six against a staging server or issuer with no human or model in the loop (S); only three (H) need a live human approver or the model. This three-way cut, sharper than a binary static/runtime line, says what a pre-deployment tool, and separately a tightened standard, can and cannot guarantee.

three ecosystems, Go (golang-jwt with keyfunc, lestrrat-go/jwx, go-jose), Node (jsonwebtoken with jwks-rsa, panva/jose), and Python (PyJWT, Authlib, python-jose), plus the Envoy `jwt_authn` proxy that fronts production service meshes. Each was configured to its standard access-token validation: verify the RS256 signature against a test issuer's JWKS, require the issuer and the audience, and require expiry where the verifier offers an option for it. A correct resource server must reject every adversarial token. We minted the token-static corpus of §4 bound to each verifier, fired it as a bearer token, and recorded acceptances. The generator and the verifiers share no code, so this is an independent test, not the generator checking its own analyzer.

Every verifier accepted the valid control, confirming it trusts the issuer. **Six of the nine then accepted a token they should have rejected** (Table 2). Four accepted a token carrying no `exp`, among them the Envoy proxy, not only libraries: these verifiers offer no way to *require* expiry in their standard configuration, so a token that never expires passes, against RFC 8725. One, python-jose, accepted a token with no bound audience even though the expected audience was supplied, which is the unbound-audience passthrough of §3; we reproduced it outside the harness. One accepted a structurally malformed token. Every other attack, algorithm confusion and the `alg:none` family, tampered and empty signatures, `kid`/`jku`/`x5u` injection, wrong issuer or audience, expired and not-yet-valid, was rejected by all nine. These are defaults, not exotic bugs, which is the systematization's point made concrete: the token-level weaknesses are the common case, and a widely-deployed proxy is among those that miss one. The run is also the mode-S conformance test of §4 in practice, separating the verifiers that enforce the token-static invariants from those that do not.

Table 2: Nine reference verifiers against the token-static corpus, standard validation config. A token accepted is a token a correct resource server must reject. Four accept a token with no exp (SCOPE-02), including the Envoy proxy; one accepts an unbound audience (MCP-01/02 passthrough); one accepts a malformed token.

Verifier	Kind	Accepted
golang-jwt + keyfunc	Go lib	—
lestrrat-go/jwx	Go lib	malformed token
go-jose/go-jose	Go lib	no-exp token
jsonwebtoken + jwks-rsa	Node lib	no-exp token
jose (panva)	Node lib	no-exp token
PyJWT	Python lib	—
Authlib	Python lib	—
python-jose	Python lib	unbound audience
Envoy `jwt_authn`	proxy	no-exp token

6 RELATED WORK

We surveyed 44 research works on authorization, delegation, identity, and token security for agents and MCP, published 2023–2026 and gathered by keyword and citation search over arXiv and the OAuth, MCP, OWASP, and CWE corpora; all arXiv identifiers were verified to exist. We coded each work for which of the seven threat categories of Table 3 it treats and at what depth (central contribution, partial, or absent), which yields the coverage scoring used below. The fifteen threats are the authorization failures these works name that are expressible as a property of a token, a key set, or a declared contract; we rate a threat *critical* when it directly grants an injected agent authority beyond its task or defeats attribution, *high* when it materially widens exposure, and *medium* when it is a precondition that enables the others rather than causing direct loss, following the severity bands of the OWASP LLM and Agentic risk ratings [32, 33]. The works divide into *threat taxonomies* and *point mechanisms*, and neither systematizes authorization by our three axes.

Taxonomies and surveys. MCP security surveys enumerate attack surfaces across the server lifecycle [14], and recent systematizations consolidate attacks and defenses for the ecosystem [9] and for multi-agent systems end to end [52]. Cross-protocol surveys map the security and interoperability of MCP, A2A, ACP, and ANP and compare their guarantees [7, 20, 24]. Threat taxonomies name authorization-adjacent risks explicitly, including the confused deputy and over-broad consent [42], and a defense-placement taxonomy classifies where in the architecture a control sits [38]. Protocol-level analyses dissect the MCP specification itself, finding missing capability attestation and implicit trust propagation across chained servers [8, 25]. These organize by attacker type, lifecycle phase, defense placement, or ecosystem component, and none sorts threats by pre-deployment testability or binds each to the clause it violates.

Delegation and identity mechanisms. A line of work makes delegation verifiable: authenticated-delegation frameworks give an agent scoped, auditable authority to act for a user [44]; Agentic JWT binds each agent action to a user intent in the token [10]; AIP carries delegation chains across MCP and A2A boundaries [34]; bounded-delegation and stateful-effect schemes keep a sub-agent within what was delegated [29, 47]; and secret-use delegation and residual authorization treat scoped secret handling and

revocation [3, 55]. Identity work gives agents a verifiable, distinct principal through WIMSE and SPIFFE/SPIRE [31, 35, 40], and full governance architectures combine registration, identity, and access tokens [46]. These are the mechanisms our delegation and identity categories draw on, each addressing a slice rather than the authorization layer as a whole.

Privilege control and model-level defenses. Runtime privilege frameworks enforce least privilege with symbolic rules and monotonic confinement [43], move the authorization decision before each tool call [48], apply mandatory access control to agent tool use [16], isolate tools in separate environments [51], and enforce policy out-of-band at a trusted boundary [27]; measurements show over-privilege is widespread in practice [22], privilege escalation through the agent's execution context has its own analysis [13], and runtime authorization governs the resources an agent acquires mid-task [57]. For the model-level threats, by-design defenses extract control and data flow so untrusted input cannot alter program flow [5], benchmarks quantify injection success [6], and tool-poisoning work measures and defends the description surface [36, 50, 54, 56], with indirect prompt injection the seminal threat [11], and recent taxonomies chart the prompt-injection threat space for LLM agents and for agentic coding assistants specifically [26, 49]. Each is one point on the static/runtime line; none draws the line, and the densest coverage is on the model-level attacks rather than the token-level guarantees.

The coverage gap, quantified. Scoring the representative works on seven threat categories (Table 3) makes the gap numeric rather than impressionistic: of the twenty-one works scored, prompt injection is a central contribution in fourteen, but audience binding is central in only three (Agentic JWT [10], aiAuthZ [19], and one enterprise framework [30]) and confused-deputy / dynamic registration in only two [30, 42], where it figures mostly as a named taxonomy entry rather than a verified or enforced property. The field has thoroughly characterized the model-level attacks and underformalized the token- and consent-level authorization guarantees, the seam this SoK occupies. This gap is a claim about the *literature*, evidenced by the survey's own coding; whether the corresponding weaknesses are also prevalent in deployed servers is a separate, empirical question we scope below.

7 DISCUSSION: WHAT A STANDARD COULD MANDATE

The testability spectrum has a direct standardization reading: the twelve testable threats are exactly the ones a profile could make mandatory-to-verify. An MCP or OAuth profile that *requires* resource indicators (RFC 8707), rejects passthrough, mandates nested `act` validation, enforces `may_act` on exchange, and caps scope and lifetime would move them from "deployer's responsibility, usually missed" to "checked at the boundary." The split tells a conformance suite how to check each: the six token-static requirements it adjudicates from a token alone, and the six server-dynamic ones by firing minted tokens, registration requests, and token exchanges at a candidate server and recording which it accepts. Table 4 makes the token-static half concrete, pairing each clause with the token minted to violate it and the expected rejection, so the profile ships with its own executable check rather than prose a deployer interprets. Only the three human-or-model threats fall outside what such a profile and suite can settle.

An agenda for the untested nine. The nine threats outside token inspection are not uniformly open; the spectrum sorts them into two agendas. Six are server-dynamic and want *engineering*: a conformance harness that fires minted tokens and registration and exchange requests at a candidate server and records what it accepts would cover whether the issuer enforces `may_act` or issues delegation rather than impersonation (DEL-03/04), whether the agent is provisioned a distinct credential (SCOPE-03), whether a deployed substrate such as SPIFFE/SPIRE [35] attests each agent (AID-01/02), and whether dynamic registration is left open (CD-02). The other three want *mechanism research*, because no server-side test settles them: per-client human consent (CD-01) needs an auditable consent record so that which client obtained which grant is reconstructable rather than trusted, and the agency threats (AGT-01/02) need a trusted mediation boundary that survives a prompt-injected plan, which the control- and data-flow work [5] approaches but does not yet reduce to a standard. Separating the engineering agenda from the research one is the practical payoff of the spectrum: it says which untested threats a conformance suite can retire and which remain genuinely open.

8 THREATS TO VALIDITY

The MCP specification is fast-moving; this systematization is a dated snapshot. The testability spectrum depends on token and manifest availability; a deployment that hides relevant state reduces what any check can conclude. The token/server/human cut is a judgment on the weakest harness per threat; a few threats sit near a boundary (for example AID-01/02 are server-dynamic only once an identity substrate is deployed in staging), which we mark rather than hide. Our survey covers research indexed by 2026-10; most agent-authorization work is recent and not yet peer-reviewed, which we mark. The coverage gap of §6 is a claim about the research literature; our evaluation (§5) gives direct evidence that the token-static weaknesses occur in widely-used verifier libraries, not only on paper. A broader prevalence measurement over a corpus of public MCP server deployments, with a hand-validated stratified sample, reported inter-rater agreement, and coordinated disclosure for identifiable high-severity findings, remains future work.

ETHICS CONSIDERATIONS

The systematization uses only public specifications, advisories, and CVEs. The accompanying analyzer runs on tokens the operator already holds and sends nothing anywhere; we probe no third-party agents or servers and use no real user tokens. Any future measurement will characterize population prevalence, not single deployments, and will follow coordinated disclosure for identifiable high-severity findings. No personal data or human subjects are involved.

OPEN SCIENCE

We release the static analyzer, a machine-readable threat taxonomy (each threat with its invariant and normative sources), and

Table 3: Coverage of the seven agent-authorization threat categories across representative works. √: a central contribution; ○: partially addressed; blank: not addressed. AudBind, audience binding and passthrough; OBO, on-behalf-of delegation; Scope, scope minimization; ConfDep, confused deputy and dynamic registration; WID, workload/agent identity; PI, prompt-injection escalation; Tool, tool poisoning. Two columns, AudBind and ConfDep, are thin.

Work	AudBind	OBO	Scope	ConfDep	WID	PI	Tool
Taxonomies and surveys							
MCP Landscape [14]	○	○	○	○	○	√	√
SoK MCP [9]	○	○	○	○	○	√	√
MCP-38 [42]	○	○	○	√	○	√	√
Enterprise MCP [30]	√	√	√	√	○	√	√
Breaking the Protocol [25]	○			○	○	√	√
Delegation and identity mechanisms							
Agentic JWT [10]	√	√	√	○	√	○	
AIP [34]	○	√	○	○	√		
Bounded Agents [29]	○	√	√	○	○	○	
AID-Guard [47]	○	√	√	○	○	○	
SAGA [46]	○	√	√	○	√	○	
AI Identity [31]	○	√	○	○	√		
aiAuthZ [19]	√	○	○	○	√		
Privilege / runtime control							
Progent [43]			√			√	○
Pre-Action Authz [48]	○	○	√	○	○	√	
MAC framework [16]		○	√	○		√	○
IsolateGPT [51]			√	○	○	√	○
If Agents Were Angels [27]	○	○	√	○	○	√	○
Model-level attacks and defenses							
Indirect PI [11]						√	○
CaMeL [5]			√	○		√	○
MCPTox [50]			○			√	√
No-Box MCP [56]						√	√

Table 4: A conformance profile for the six token-static threats. Each row is a requirement a profile could mandate and a test a suite runs from a token, a key set, and a declared contract, with no running agent. The six server-dynamic threats extend this with tests fired at a candidate server.

Threat	Clause	Conformance test
MCP-01/02	RFC 8707	Mint a token whose `aud` names another resource; the verifier must reject it.
DEL-01/02	RFC 8693	Present a delegated token with a missing or malformed `act` chain; the verifier must reject it.
SCOPE-01	RFC 9700	Present a token whose scope exceeds the tool's declared need; the contract check must flag it.
SCOPE-02	RFC 8725	Present a token past its `exp` or beyond a capped lifetime; the verifier must reject it.

the evaluation harness with its nine-verifier target set and raw results, under an open licence, so both the systematization and the evaluation of §5 are reproducible. A DOI-pinned artifact will accompany the camera-ready version.

9 CONCLUSION

Agent authorization is a high-stakes surface the research literature has studied around rather than through. We systematize it by pre-deployment testability, by normative provenance, and by a three-way enforcement spectrum; we show that twelve of fifteen threats are testable before production, six from a token alone and six against a staging server with no human or model in the loop, and only three need a live human approver or the model itself; we evaluate the token-static subset against nine reference verifiers and find six accept a token they should reject, so the weaknesses are defaults rather than corner cases; and we argue this cut should guide what a pre-deployment tool checks, what a conformance suite exercises, and what a standard makes mandatory to verify.

LLM USAGE STATEMENT

LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality. An LLM-assisted process additionally supported the literature survey of §6: candidate works were gathered with model assistance and each was then verified by the authors to exist, with titles and author lists confirmed against the arXiv API on 2026-10-06; works that could not be verified were excluded, and one paper that its own authors had formally withdrawn was identified this way and removed. The authors are responsible for the selection of works, the coverage judgments in the systematization, and every claim in the paper. One reproducibility limitation applies: the assisting models are not openly available, so the survey's search recall cannot be reproduced exactly, although every cited work is independently verifiable by its arXiv identifier.

A WORKED TOKEN-INSPECTION EXAMPLES

Because the mode-one checks are definitional, their verdicts are deterministic rather than learned. Table 5 gives illustrative adversarial tokens (claims abridged) and the analyzer's token-inspection verdict against a fixed policy that requires the audience `api.pay`,

a scope within the allow-list {files:read}, an exp within fifteen minutes, and a well-formed act claim on any token that asserts delegation. The examples demonstrate what the analyzer *verifies* and, in the last row, what it does not: a passing token-inspection verdict does not attest that the deployed verifier enforces the same constraints, which is conformance testing (§4). This is a demonstration of the checks, not an empirical evaluation; measuring how often real deployments carry such tokens is the prevalence study scoped in §8.

Table 5: Illustrative token-inspection verdicts against the fixed policy of §A. Verdicts are deterministic because the checks are definitional.

ID	Token (abridged)	Verdict
A1	aud=api.other	FAIL MCP-02: aud names another resource
A2	scope=admin:*	FAIL SCOPE-01: scope exceeds declared need
A3	no exp	FAIL SCOPE-02: no lifetime bound
A4	act absent, asserts OBO	FAIL DEL-01: delegation hygiene, act missing
A5	all constraints met	PASS inspection; does not attest server enforcement

REFERENCES

[1] Anthropic and MCP contributors. 2025. Model Context Protocol: Authorization and Security Best Practices. MCP Specification 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18/basic/security_best_practices.

[2] B. Campbell, J. Bradley, and H. Tschofenig. 2020. Resource Indicators for OAuth 2.0. RFC 8707, IETF. https://datatracker.ietf.org/doc/html/rfc8707.

[3] Moonwon Choi, Seokho Jeong, Sian Choi, and Seunggeun Lee. 2026. ResidualAuth: What Authorization State Must Language Agents Preserve under Revocable Delegation? arXiv:2609.08062 [cs.CR] arXiv preprint. https://arxiv.org/abs/2609.08062.

[4] Panduranga Sai Varma Dantuluri and Jyotirmoy Sundi. 2026. Delegation Without Trust: An Empirical Gap Analysis of Identity, Authorization, and Runtime Governance in Multi-Agent LLM Systems. arXiv:2609.00267 [cs.CR] arXiv preprint. https://arxiv.org/abs/2609.00267.

[5] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating Prompt Injections by Design. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. CaMeL; also arXiv:2503.18813. https://arxiv.org/abs/2503.18813.

[6] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. Also arXiv:2406.13352. https://arxiv.org/abs/2406.13352.

[7] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). arXiv:2505.02279 [cs.MA] arXiv preprint. https://arxiv.org/abs/2505.02279.

[8] Yuval Felendler, Parth A. Gandhi, Idan Habler, Yuval Elovici, and Asaf Shabtai. 2026. From Tool Orchestration to Code Execution: A Study of MCP Design Choices. arXiv:2602.15945 [cs.CR] arXiv preprint. https://arxiv.org/abs/2602.15945.

[9] Shiva Gaire, Srijan Gyawali, Saroj Mishra, Suman Niroula, Dilip Thakur, and Umesh Yadav. 2025. Systematization of Knowledge: Security and Safety in the Model Context Protocol Ecosystem. arXiv:2512.08290 [cs.CR] arXiv preprint. https://arxiv.org/abs/2512.08290.

[10] Abhishek Goswami. 2025. Agentic JWT: A Secure Delegation Protocol for Autonomous AI Agents. arXiv:2509.13597 [cs.CR] arXiv preprint. https://arxiv.org/abs/2509.13597.

[11] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*. Also arXiv:2302.12173. https://arxiv.org/abs/2302.12173.

[12] Yongjian Guo, Puzhuo Liu, Wanlun Ma, Zehang Deng, Xiaogang Zhu, Peng Di, Xi Xiao, and Sheng Wen. 2025. MCPXKIT: The Unified Toolkit for Analyzing Model Context Protocol Security. *IEEE Transactions on Dependable and Secure Computing (TDSC)* (2025). Accepted; also arXiv:2508.12538. https://arxiv.org/abs/2508.12538.

[13] Xingbang He, Yuanwei Chen, Yi Qian, Haiyang Wei, Ligeng Chen, Zenan Fu, Linzhang Wang, Hao Wu, and Bing Mao. 2026. When Context Gets Root: Privilege Escalation in LLM Harnesses. arXiv:2608.27299 [cs.CR] arXiv preprint. https://arxiv.org/abs/2608.27299.

[14] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv:2503.23278 [cs.CR] arXiv preprint; widely cited. https://arxiv.org/abs/2503.23278.

[15] IETF WIMSE WG. 2025. Workload Identity Practices (WIMSE). Internet-Draft. https://datatracker.ietf.org/doc/draft-ietf-wimse-workload-identity-practices/.

[16] Zimo Ji, Daoyuan Wu, Wenyuan Jiang, Pingchuan Ma, Zongjie Li, Yudong Gao, Shuai Wang, and Yingjiu Li. 2026. Taming Various Privilege Escalation in LLM-Based Agent Systems: A Mandatory Access Control Framework. arXiv:2601.11893 [cs.CR] arXiv preprint. https://arxiv.org/abs/2601.11893.

[17] M. Jones, P. Hunt, and A. Parecki. 2025. OAuth 2.0 Protected Resource Metadata. RFC 9728, IETF. https://datatracker.ietf.org/doc/html/rfc9728.

[18] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore. 2020. OAuth 2.0 Token Exchange. RFC 8693, IETF. https://datatracker.ietf.org/doc/html/rfc8693.

[19] Sai Varun Kodathala. 2026. aiAuthZ: Off-Host, Identity-Bound Authorization for AI Agents. arXiv:2607.05518 [cs.CR] Technical report. https://arxiv.org/abs/2607.05518.

[20] Dezhang Kong, Shi Lin, Zhenhua Xu, Zhebo Wang, Minghao Li, Yufeng Li, Yilun Zhang, Hujin Peng, Xiang Chen, Zeyang Sha, Yuyuan Li, Changting Lin, Xun Wang, Xuan Liu, Ningyu Zhang, Chaochao Chen, Chunming Wu, Muhammad Khurram Khan, and Meng Han. 2025. A Survey of LLM-Driven AI Agent Communication: Protocols, Security Risks, and Defense Countermeasures. arXiv:2506.19676 [cs.CR] arXiv preprint. https://arxiv.org/abs/2506.19676.

[21] Qiaomu Li and Ying Xie. 2025. From Glue-Code to Protocols: A Critical Analysis of A2A and MCP Integration for Scalable Agent Systems. arXiv:2505.03864 [cs.SE] arXiv preprint. https://arxiv.org/abs/2505.03864.

[22] Zhihao Li, Kun Li, Boyang Ma, Minghui Xu, Yue Zhang, and Xiuzhen Cheng. 2025. We Urgently Need Privilege Management in MCP: A Measurement of API Usage in MCP Ecosystems. arXiv:2507.06250 [cs.CR] arXiv preprint. https://arxiv.org/abs/2507.06250.

[23] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett. 2025. Best Current Practice for OAuth 2.0 Security. RFC 9700, IETF. https://datatracker.ietf.org/doc/html/rfc9700.

[24] Yedidel Louck, Ariel Stulman, and Amit Dvir. 2025. Security Analysis of Agentic AI Communication Protocols: A Comparative Evaluation. arXiv:2511.03841 [cs.CR] arXiv preprint. https://arxiv.org/abs/2511.03841.

[25] Narek Maloyan and Dmitry Namiot. 2026. Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents. arXiv:2601.17549 [cs.CR] arXiv preprint. https://arxiv.org/abs/2601.17549.

[26] Narek Maloyan and Dmitry Namiot. 2026. Prompt Injection Attacks on Agentic Coding Assistants: A Systematic Analysis of Vulnerabilities in Skills, Tools, and Protocol Ecosystems. arXiv:2601.17548 [cs.CR] arXiv preprint. https://arxiv.org/abs/2601.17548.

[27] Marc Millstone, Tyler Akidau, Johannes Brüderl, and Marat Pekker. 2026. If Agents Were Angels, No Governance Would Be Necessary: Out-of-Band Policy Enforcement at a Trusted Tool Boundary. arXiv:2608.27646 [cs.CR] arXiv preprint. https://arxiv.org/abs/2608.27646.

[28] MITRE. 2024. CWE-441: Unintended Proxy or Intermediary (Confused Deputy). https://cwe.mitre.org/data/definitions/441.html.

[29] Xabier Muruaga. 2026. Bounded Agents: Delegation Security for Multi-Agent AI Systems. arXiv:2608.15888 [cs.CR] arXiv preprint. https://arxiv.org/abs/2608.15888.

[30] Vineeth Sai Narajala and Idan Habler. 2025. Enterprise-Grade Security for the Model Context Protocol (MCP): Frameworks and Mitigation Strategies. arXiv:2504.08623 [cs.CR] arXiv preprint. https://arxiv.org/abs/2504.08623.

[31] Takumi Otsuka, Kentaroh Toyoda, and Alex Leung. 2026. AI Identity: Standards, Gaps, and Research Directions for AI Agents. arXiv:2604.23280 [cs.CR] arXiv preprint. https://arxiv.org/abs/2604.23280.

[32] OWASP. 2025. OWASP Top 10 for Large Language Model Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/.

[33] OWASP GenAI Security Project. 2025. OWASP Top 10 Risks & Mitigations for Agentic AI. https://genai.owasp.org/.

[34] Sunil Prakash. 2026. AIP: Agent Identity Protocol for Verifiable Delegation Across MCP and A2A. arXiv:2603.24775 [cs.CR] arXiv preprint. https://arxiv.

org/abs/2603.24775.
[35] Sunil Prakash. 2026. LDP: An Identity-Aware Protocol for Multi-Agent LLM Systems. arXiv:2603.08852 [cs.CR] arXiv preprint. https://arxiv.org/abs/2603.08852.
[36] Mohammadreza Rashidi. 2026. Unicode TAG-Block Concealment of Tool-Metadata Payloads in the Model Context Protocol: An Approval-View Fidelity Gap Across Three Independent Server Implementations. arXiv:2607.05744 [cs.CR] arXiv preprint. https://arxiv.org/abs/2607.05744.
[37] J. Richer, M. Jones, J. Bradley, M. Machulak, and P. Hunt. 2015. OAuth 2.0 Dynamic Client Registration Protocol. RFC 7591, IETF. https://datatracker.ietf.org/doc/html/rfc7591.
[38] Mehrdad Rostamzadeh, Sidhant Narula, Nahom Birhan, Mohammad Ghasemigol, and Daniel Takabi. 2026. MCP-DPT: A Defense-Placement Taxonomy and Coverage Analysis for Model Context Protocol Security. arXiv:2604.07551 [cs.CR] arXiv preprint. https://arxiv.org/abs/2604.07551.
[39] Christian Schroeder de Witt, Klaudia Krawiecka, Igor Krawczuk, et al. 2025. Open Challenges in Multi-Agent Security: Towards Secure Systems of Interacting AI Agents. arXiv:2505.02077 [cs.MA] arXiv preprint (24 authors). https://arxiv.org/abs/2505.02077.
[40] Riddhi Mohan Sharma. 2026. Ethical Hyper-Velocity (EHV): A Hardware-Rooted Zero-Trust Runtime Enforcement Architecture for Agentic AI Systems. arXiv:2605.17909 [cs.CR] arXiv preprint; integrates SPIFFE/SPIRE. https://arxiv.org/abs/2605.17909.
[41] Y. Sheffer, D. Hardt, and M. Jones. 2020. JSON Web Token Best Current Practices. RFC 8725, IETF. https://datatracker.ietf.org/doc/html/rfc8725.
[42] Yi Ting Shen, Kentaroh Toyoda, and Alex Leung. 2026. MCP-38: A Comprehensive Threat Taxonomy for Model Context Protocol Systems (v1.0). arXiv:2603.18063 [cs.CR] arXiv preprint. https://arxiv.org/abs/2603.18063.
[43] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703 [cs.CR] arXiv preprint; widely cited. https://arxiv.org/abs/2504.11703.
[44] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. 2025. Authenticated Delegation and Authorized AI Agents. arXiv:2501.09674 [cs.CR] ICML 2025. https://arxiv.org/abs/2501.09674.
[45] SPIFFE Project. 2024. SPIFFE: Secure Production Identity Framework for Everyone. https://spiffe.io/.
[46] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. 2025. SAGA: A Security Architecture for Governing AI Agentic Systems. arXiv:2504.21034 [cs.CR] arXiv preprint. https://arxiv.org/abs/2504.21034.
[47] Yingzhe Tong, Leyu Dai, and Songhui Guo. 2026. AID-Guard: Stateful Authorization for Delegated Agent Effects. arXiv:2608.21159 [cs.CR] arXiv preprint. https://arxiv.org/abs/2608.21159.
[48] Uchi Uchibeke. 2026. Before the Tool Call: Deterministic Pre-Action Authorization for Autonomous AI Agents. arXiv:2603.20953 [cs.CR] arXiv preprint. https://arxiv.org/abs/2603.20953.
[49] Peiran Wang, Xinfeng Li, Chong Xiang, Jinghuai Zhang, Ying Li, Lixia Zhang, Xiaofeng Wang, and Yuan Tian. 2026. The Landscape of Prompt Injection Threats in LLM Agents: From Taxonomy to Analysis. arXiv:2602.10453 [cs.CR] arXiv preprint. https://arxiv.org/abs/2602.10453.
[50] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2025. MCP-Tox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. arXiv:2508.14925 [cs.CR] arXiv preprint. https://arxiv.org/abs/2508.14925.
[51] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Proceedings of the Network and Distributed System Security (NDSS) Symposium*. Also arXiv:2403.04960. https://arxiv.org/abs/2403.04960.
[52] Rui Yang, Junjie Xu, Zhengyu Liu, Neil Fendley, Yang Hong, Ziyang Li, and Yinzhi Cao. 2026. SoK: When Safe Agents Fail Together: The Security of Multi-Agent LLM Systems. arXiv:2609.00595 [cs.CR] arXiv preprint. https://arxiv.org/abs/2609.00595.
[53] Yixuan Yang, Cuifeng Gao, Daoyuan Wu, Yufan Chen, Yingjiu Li, and Shuai Wang. 2025. MCPSecBench: A Systematic Security Benchmark and Playground for Testing Model Context Protocols. arXiv:2508.13220 [cs.CR] Technical report, Lingnan University. https://arxiv.org/abs/2508.13220.
[54] Hengkai Ye, Zhechang Zhang, Jinyuan Jia, and Hong Hu. 2026. TRUSTDESC: Preventing Tool Poisoning in LLM Applications via Trusted Description Generation. arXiv:2604.07536 [cs.CR] arXiv preprint. https://arxiv.org/abs/2604.07536.
[55] Xiaohang Yu, Hejia Geng, Xinmeng Zeng, and William Knottenbelt. 2026. SUDP: Secret-Use Delegation Protocol for Agentic Systems. arXiv:2604.24920 [cs.CR] arXiv preprint. https://arxiv.org/abs/2604.24920.
[56] Zehua Zhang, Jie Hu, Pratham Hegde, Aditya Maheshbhai Gabani, Souradip Nath, Yibo Liu, Siyu Liu, Hongkai Chen, Hulin Wang, Zhuoer Lyu, Chang Zhu, Divij Handa, Yan Shoshitaishvili, Tiffany Bao, Ruoyu Wang, and Adam Doupé. 2026. No-Box Vulnerability Analysis: Description-only Detection of Indirect Prompt Injection Vulnerabilities in MCP Servers. arXiv:2609.10854 [cs.CR] arXiv preprint. https://arxiv.org/abs/2609.10854.
[57] Genliang Zhu and Chu Wang. 2026. Runtime Authorization for Resources Acquired by AI Agents. arXiv:2609.14744 [cs.CR] arXiv preprint. https://arxiv.org/abs/2609.14744.